



เอกสารเตรียมนำเสนอโครงการ (ฉบับอ่านก่อนสอบ)

เกมการ์ดเสริมสร้างแนวคิดระบบสารสนเทศ — *Startup City / The Great Hackathon*

วิธีใช้เอกสารนี้: อ่านจากบนลงล่างได้เลย ส่วนที่สำคัญที่สุด 3 ส่วนคือ 1. **ส่วนที่ 1 (ตัวเลขที่ต้องจำ)** — ท่องให้ขึ้นใจ อาจารย์ชอบถามตัวเลข 2. **ส่วนที่ 7 (คลังคำถาม-คำตอบ)** — ซ้อมตอบให้คล่อง นี่คือหัวใจของการสอบ 3. **ส่วนที่ 8 (จุดที่เอกสารกับโค้ดไม่ตรงกัน)** — ⚠️ ต้องรีบแก้ก่อนสอบ ไม่งั้นโดนจับผิด

เอกสารนี้สรุปจาก "โค้ดจริง" ในโปรเจกต์ ไม่ใช่แค่จากเล่ม ดังนั้นเชื่อถือได้ว่าตรงกับของที่รันจริง

🎯 ส่วนที่ 0 — สรุป 1 นาที (พูดได้ทันทีถ้าอาจารย์บอก "เล่าโปรเจกต์ให้ฟังหน่อย")

"โครงการนี้คือ **เกมการ์ดดิจิทัลเชิงการศึกษาบนมือถือ** ที่นำกลไกการบริหารทรัพยากรของบอร์ดเกม **Splendor** มาประยุกต์ใหม่ให้เป็นการจัดการ **ทรัพยากรระบบคอมพิวเตอร์ 5 อย่าง** คือ CPU, RAM, Network, Storage และ Security ผู้เล่นสวมบทเป็นนักพัฒนาระบบ ใช้ทรัพยากรเหล่านี้ไปสร้าง 'การ์ดโมดูลซอฟต์แวร์' 3 ระดับ เพื่อสะสมแต้ม (Stars) ให้ครบ **20 แต้มเป็นคนแรก**

จุดที่ทำให้เป็น 'สื่อการเรียนรู้' ไม่ใช่แค่เกม คือมี **ควิซความรู้คอมพิวเตอร์แทรกระหว่างเล่น** เพื่อชิงสิทธิ์ลำดับการเดินในรอบถัดไป

ในเชิงเทคนิค ผมพัฒนาเป็นระบบ **Full-Stack** จริง: ตัวเกมทำด้วย **Unity (C#)**, เล่นออนไลน์ 2-4 คนแบบเรียลไทม์ด้วย **Photon Fusion**, มีโหมดเล่นคนเดียวสู้กับ **บอท AI แบบ Rule-Based**, และเชื่อมหลังบ้านคลาวด์ **Supabase (PostgreSQL)** สำหรับสมัครสมาชิก ระบบจับคู่อัตโนมัติ คะแนนอันดับ MMR และร้านค้า"

(พูดประมาณนี้ ~45 วินาที — จำ flow: เกมอะไร → สอนอะไร → ทำด้วยอะไร)

1234 ส่วนที่ 1 — ตัวเลข & ข้อเท็จจริงที่ต้องจำขึ้นใจ (Cheat Sheet)

หัวข้อ	ค่า	หมายเหตุ
เป้าหมายชนะ	20 แด้ม (Stars)	คนแรกถึงเป็นผู้ชนะ
ทรัพยากร	5 ชนิด	CPU, RAM, Network, Storage, Security
เหรียญพิเศษ	Gold / Wildcard	ใช้แทนสีใดก็ได้ ได้จาก "การจองการ์ด" เท่านั้น
ระดับการ์ด	3 Tiers	T1: 0–1 แด้ม / T2: 1–3 แด้ม / T3: 3–5 แด้ม
ขุนนาง (Nobles)	8 ใบ	ได้อัตโนมัติเมื่อมีโบนัสการ์ดครบเงื่อนไข (ให้แด้มเพิ่ม)
แอคชั่นต่อเทิร์น	เลือก 1 จาก 3	หยิบเหรียญ / ซื้อการ์ด / จองการ์ด
เพดานถือเหรียญ	10 เหรียญ	เกินไม่ได้ ต้องซื้อการ์ดเพื่อใช้เหรียญก่อน
จองการ์ดได้สูงสุด	3 ใบ	ได้ Wildcard +1 ต่อการจอง
เหรียญในกองกลาง	2 คน=4, 3 คน=5, 4 คน=7 ต่อสี	Gold = 5 เสมอ
กฎหยิบเหรียญ	3 สีต่างกัน หรือ 2 สีเดียวกัน	หยิบ 2 สีเดียวได้เมื่อกองนั้นเหลือ ≥ 4
เวลาต่อเทิร์น	30 วินาที	หมดเวลา = บังคับข้ามเทิร์น
เวลาตอบควิซ	10 วินาที	⚠ เล่ม 3.5 เขียน 20 วิ — ดูส่วนที่ 8
ควิซแทรกทุกๆ	⚠ โค้ดจริง = 5 รอบ	เล่มเขียน 3 รอบ — ต้องแก้ ดูส่วนที่ 8
บอทดตอบควิซถูก	สุ่ม 40–80%	⚠ เล่มเขียน 100% — ดูส่วนที่ 8
ผู้เล่นออนไลน์	2–4 คน	Photon Fusion (Shared Mode)
โหมดบอท	Rule-Based 4 ลำดับ + 4 บุคลิก	ไม่ใช่ Machine Learning
MMR เริ่มต้น	1000	สูตรแบบ DOTA2 (ไม่ใช่ Elo)
รางวัล Gem ตามอันดับ	1=5, 2=3, 3=2, 4=1	เพชรเอาไปซื้อกรอบโปรไฟล์
Daily Quiz	วันละ 1 ครั้ง, 5 ข้อ	กันรับซ้ำด้วยตาราง <code>daily_quiz_claims</code>
ตารางฐานข้อมูล	8 ตาราง	ดูส่วนที่ 4.6

หัวข้อ	ค่า	หมายเหตุ
ผลประเมินความพึงพอใจ	4.50 (S.D. 0.54)	จากผู้ทดลอง 15 คน, Likert 5 ระดับ = "มากที่สุด"
Engine / เวอร์ชัน	Unity 6 (6000.x)	ภาษา C#



ส่วนที่ 2 — สถาปัตยกรรมระบบ (อธิบายให้เห็นภาพ)

โปรเจกต์นี้เป็น **3 ชั้น (3-Tier Full-Stack)** — ถ้าอาจารย์ถาม "ระบบทำงานยังไง" ให้วาดภาพนี้



ประโยชน์ที่ผู้ทดสอบ:

"ตัวเกมคุยกับหลังบ้าน 2 ทาง — เรื่อง **เรียลไทม์ระหว่างเล่น** (สถานะกระดาน, เทิร์น) วิ่งผ่าน **Photon Fusion** บน UDP เพราะต้องการความหน่วงต่ำ ส่วนเรื่อง **ข้อมูลถาวร** (บัญชี, คะแนน, ร้านค้า, ข้อสอบ) วิ่งผ่าน **Supabase** ผมแยก 2 ระบบนี้ตามลักษณะงาน — อะไรที่ต้องเร็วและชั่วคราวใช้ Fusion อะไรที่ต้องเก็บถาวรและเชื่อถือได้ใช้ Supabase"



ส่วนที่ 3 — กลไกเกมแบบละเอียด (กติกาที่ต้องอธิบายได้)

3.1 ทรัพยากร & เหรียญ

- เหรียญทรัพยากร 5 สี = **CPU, RAM, Network, Storage, Security**
- เหรียญ **Gold (Wildcard)** = แทนสีใดก็ได้ — ห้ามหยิบตรงๆ ได้จากการ "จองการ์ด" เท่านั้น
- ถือเหรียญรวมกันได้ไม่เกิน **10 อัน**

3.2 การ์ดโมดูลซอฟต์แวร์ (Engine-Building)

- การ์ดแต่ละใบมี **ต้นทุน** (เหรียญที่ต้องจ่าย) และ **แต้ม (Stars)**
- ซื้อการ์ดแล้วได้ **โบนัสถาวร 1 สี** = ส่วนลดต้นทุนการ์ดใบต่อไป (นี่คือหัวใจ "Engine-Building" ของ Splendor — ยิ่งซื้อยิ่งถูกลง)
- มี 3 Tier เรียงจากถูก/แต้มน้อย (T1) ไปแพง/แต้มเยอะ (T3)

3.3 แอคชั่นในแต่ละเทิร์น (เลือกได้ 1 อย่าง)

1. **หยิบเหรียญ (Collect):** 3 สีต่างกัน หรือ 2 สีเดียวกัน (ถ้ากองนั้นเหลือ ≥ 4)
2. **ซื้อการ์ด (Purchase):** จ่ายเหรียญตามต้นทุน (หักโบนัสที่มีก่อน) $\rightarrow$ ได้แต้ม + โบนัส
3. **จองการ์ด (Reserve):** เก็บการ์ดขึ้นมือ(สูงสุด 3 ใบ) + ได้ Gold 1 เหรียญ — ใช้ตัดหน้าคู่แข่ง/ล็อกการ์ดแพง

3.4 ขุนนาง (Nobles)

- มี 8 ใบ ระบบ **เช็ดให้หมดโน้มนำทุกจบเทิร์น** — ถ้าผู้เล่นมีโบนัสการ์ดครบเงื่อนไข ขุนนางจะมาหาเองและให้แต้มเพิ่ม

3.5 เงื่อนไขชนะ

- ใครถึง **20 แต้ม** ก่อน = ชนะ
- ⚠ **จุดที่ต้องระวัง:** เล่ม (Scope 3.4) เขียนว่า "เล่นให้จบรอบนั้นก่อนเพื่อให้ทุกคนเดินเท่ากัน" แต่ **โค้ดจริงจบเกมทันทีที่มีคนถึง 20** — ดูส่วนที่ 8 ข้อ 5

3.6 ควิซแทรก (จุดเด่นด้านการศึกษา)

- ทุกๆ N รอบ เกมจะหยุดแล้วตั้งคำถามคอมพิวเตอร์ 1 ข้อ (4 ตัวเลือก) ให้ทุกคนตอบพร้อมกัน จับเวลา 10 วินาที
- **ตอบถูก + เร็วที่สุด = ได้เดินเป็นคนแรกในรอบถัดไป** (เป็นการเอาความรู้มาแปลงเป็นความได้เปรียบในเกม $\rightarrow$ จูงใจให้ตั้งใจเรียน)

⚙️ ส่วนที่ 4 — เจาะระบบสำคัญ (เนื้อหาที่ใช้ฟรีเชนต์ + ตอบคำถามเชิงลึก)

4.1 บอท AI แบบ Rule-Based (โหมดเล่นคนเดียว) ★ จุดขาย

บอท **ไม่ใช่ Machine Learning** แต่เป็น **ระบบตรรกะเชิงกฎเกณฑ์ (Rule-Based)** ที่มีลำดับการตัดสินใจ 4 ชั้น ทุกเทิร์นบอทจะไล่เช็คตามลำดับนี้ เจออันไหนทำได้ก่อนทำเลย:

ลำดับ	เงื่อนไข	การกระทำ
1. ปิดเกม	มีการ์ดที่ซื้อแล้วทำให้แต้มรวม ≥ 20	ซื้อทันที (ชนะ)
2. ซื้อใบที่คุ้มสุด	มีการ์ดที่ซื้อไหว	ให้คะแนน = <code>แต้ม × น้ำหนัก - ต้นทุน</code> แล้วเลือกใบดีที่สุด
3. จองเชิงรุก	ขาดเหรียญซื้อ T3 อยู่คนเดียว	จongsล็อกการ์ดไว้ + ได้ Gold
4. เก็บของ	ทำ 1-3 ไม่ได้	หยิบเหรียญสีที่ "ต้องการมากที่สุด" บนกระดาน

ของเด็ดที่ควรรอด: บอทมี **4 บุคลิก (Archetype)** คงที่ตามทีนี้ ทำให้ตัดสินใจต่างกันแม้เจอสถานการณ์เดียวกัน: -

Balanced — มาตรฐาน - **Aggressive** — เน้นการ์ดแต้มสูง ไม่ค่อยจอง (น้ำหนักแต้ม $\times 2.0$) - **Hoarder** — ชอบจอง/สะสมทอง เลือกการ์ดถูก - **Specialist** — โฟกัสเก็บ 2 สีที่ถนัด

มี "ความไม่แน่นอน" (suboptimalChance) ให้บอทบางครั้งเลือกตัวเลือกอันดับรอง → ไม่เล่นซ้ำซากเป็นหุ่นยนต์เกินไป

ทำไมไม่เลือก Rule-Based ไม่ใช่ ML? → ตอบในส่วนที่ 7 (คำถามฮิต)

4.2 คริซ & การชิงเทิร์น

- Host (Master Client) เป็นคนสุ่มคำถามแล้ว **broadcast** ให้ทุกเครื่องเห็นข้อเดียวกัน (กันโกง/กันเห็นไม่ตรงกัน)
- ระบบวัดทั้ง **ความถูก** และ **ความเร็วระดับเสี้ยววินาที** แล้วจัดลำดับเทิร์นใหม่ (`ApplyNewTurnOrder`)
- ในโหมดเล่นคนเดียว ระบบ **จำลองคำตอบของบอท** (สุ่มถูก 40–80%, สุ่มความเร็ว 1–10 วิ) เพื่อให้มีคู่แข่งชิงเทิร์น

4.3 Matchmaking + การกัน Race Condition ★ จุดขายเชิง DB

- ผู้เล่นกด "หาคู่" → client ยิงขึ้น **Supabase Edge Function** ชื่อ `matchmaking` → ฝั่ง server เรียก **Database Function** `claim_matchmaking_room`
- ปัญหาที่แก้:** ถ้า 2 คนกดพร้อมกัน อาจถูกจับเข้าคนละห้อง/ห้องซ้ำ (Race Condition)

- **วิธีแก้ (พูดให้อาจารย์ฟังได้):** 1. ใช้ `SELECT ... FOR UPDATE` ล็อกแถวระดับธุรกรรม (transaction) 2. ใช้ **Unique Partial Index** บน `matchmaking_queue(player_id) WHERE status='waiting'` → 1 คนมีได้แค่ 1 คิว เป็นไปไม่ได้ที่จะซ้ำ 3. client poll สถานะทุก 1.5 วิ จนได้ห้อง แล้วเข้าร่วม Photon session เดียวกัน
- มี timeout กันคิวค้าง (stale) ที่ 300 วินาที

4.4 การซิงค์สถานะผ่าน Photon Fusion (Shared Mode)

- **Shared Mode** = ไม่มีเซิร์ฟเวอร์กลางตัวเดียว แต่มี **Master Client** ทำหน้าที่เป็น "ผู้ตัดสิน" (authority) เรื่องกระดาน/คิริช
- Host จะ **broadcast 3 snapshot**: สถานะกระดาน (Board), เศรษฐกิจ/เหรียญ (Economy), ลำดับเทิร์น (Turn)
- ★ **เทคนิคแก้บั๊กที่เจอจริง (พูดได้เลยว่าเจอปัญหานี้แล้วแก้):** ผู้เล่นที่เข้าห้องช้า (late-joiner) จะ **"ดึง" สถานะ** เต็มจาก **host เอง (RequestFullState)** แทนที่จะรอ host "ส่ง" มา — เพราะตอนเพิ่งเข้า ฉากอาจโหลดไม่เสร็จ ทำให้ broadcast หลุดและเห็นกระดานไม่ตรงกัน → เปลี่ยนจาก **push เป็น pull** เพื่อกัน desync
- ถ้าผู้เล่นหลุดกลางเทิร์น/หมดเวลา → `ForceEndTurn()` ให้ host บังคับจบเทิร์นแทน ไม่ให้เกมค้าง

4.5 ความปลอดภัย & Anti-Cheat ★ จุดขายที่อาจารย์ชอบ

- **Server-Authoritative:** ตอนจบเกม client ส่งขึ้น DB แค่ **"อันดับที่ได้ + จำนวนผู้เล่น"** เท่านั้น แล้ว **server** คำนวณ **MMR/Gems** เอง → client **ปลอมค่า MMR/เพชรลง DB ไม่ได้**
- **RLS (Row Level Security):** ตาราง `player_profiles` ตั้งให้ client (authenticated) **อ่านได้อย่างเดียว** เขียนได้เฉพาะ `service_role` ผ่าน Edge Function
- **ราคาร้านค้าอยู่ที่ server:** ตาราง `shop_items` เก็บราคาจริง → client แก้อาณาเขตไม่ได้
- **OTP ยืนยันอีเมล:** เก็บเป็น **SHA-256 hash** ไม่เก็บเลขตรง มี `expires_at` (หมดอายุ) และ `attempts` (จำกัดจำนวนครั้งเดา)

4.6 ฐานข้อมูล (Supabase / PostgreSQL) — 8 ตาราง

ตาราง	หน้าที่
player_profiles	โปรไฟล์หลัก: gems, mmr, wins, losses, กรอบที่มี/ที่ใส่, ตัวละคร (1:1 กับ auth.users)
rooms	ห้องเกมที่จับคู่สร้าง (ผูกกับ Photon session)
matchmaking_queue	คิวจับคู่ออนไลน์ (มี RPC claim_matchmaking_room)
shop_items	ราคาไอเทมร้านค้า (source of truth กันโกงราคา)
daily_quiz_claims	กันรับรางวัลควิซรายวันซ้ำ (1 คน/วัน)
quiz_patches	ชุด/แพ็กข้อสอบ (เปิด-ปิด active ได้)
quiz_questions	คลังคำถาม (โจทย์, 4 ตัวเลือก jsonb, เฉลย) ผูกกับ patch
otp_codes	รหัส OTP ยืนยันอีเมล (เก็บ hash)

จดขาย: ระบบควิซเป็น **"Patch System"** — เพิ่ม/แก้ไขข้อสอบในฐานข้อมูลได้ **โดยไม่ต้อง build เกมใหม่** เพราะตัวเกมดึงคำถามจาก patch ที่ active ผ่าน RPC

4.7 ระบบ MMR (จัดอันดับ)

- เริ่มที่ **1000** สูตรแบบ **DOTA2** (เป็นค่าคงที่ตามอันดับ ไม่ใช่ Elo ที่คิดจากส่วนต่างคะแนนคู่แข่ง)

จำนวนผู้เล่น	อันดับ 1	2	3	4
2 คน	+25	-25	—	—
3 คน	+25	-5	-20	—
4 คน	+30	+10	-10	-25

- ยศ: Bronze < Silver(1000) < Gold(1500) < Platinum(2000) < Diamond(2500) < Legend(3000)

 ส่วนที่ 5 — จดขายเชิงเทคนิค (สิ่งที่ทำให้อาจารย์ประทับใจ)

ถ้าอาจารย์ถาม "โปรเจกต์นี้เด่นกว่าการเอาเกม Splendor มาทำใหม่ตรงไหน" ให้ชู 6 ข้อนี้:

- 1. เป็น Full-Stack จริง — ไม่ได้หยุดที่ตัวเกม แต่มี backend คลาวด์ + ระบบสมาชิก + จัดอันดับ + ร้านค้า ครบวงจร
- 2. Multiplayer เร็วใหม่ ที่จัดการ desync/late-join/ผู้เล่นหลุด ได้จริง (ไม่ใช่แค่ทฤษฎี)
- 3. กัน Race Condition ระดับฐานข้อมูล ด้วย row-lock + unique partial index
- 4. Server-Authoritative + RLS กันโกงค่าสถิติและราคา
- 5. บอท AI ที่มีลำดับการตัดสินใจชัดเจน + บุคลิกหลากหลาย + ความไม่แน่นอนแบบมนุษย์
- 6. บุรณาการการศึกษาเข้ากับกลไกเกม — ความรู้ = ความได้เปรียบ (ควิซชิงเทิร์น) ไม่ใช่แค่แปะคำถามเฉยๆ

 ส่วนที่ 6 — สคริปต์การนำเสนอ (ลำดับ + เวลาโดยประมาณ)

ปรับเวลาตามที่อาจารย์กำหนด ตัวอย่างนี้คือ ~12-15 นาที

นาที	สไลด์/หัวข้อ	พูดอะไร / โชว์อะไร
0:00-1:00	เปิด + ที่มา	ปัญหา: เรียน IT ทองจ่านาเปื่อ → แก้ด้วยเกม + ควิซ (ใช้บท intro)
1:00-2:00	วัตถุประสงค์	3 ข้อ: ทำเกมการศึกษา / ทำบอท Rule-Based / ออกแบบฐานข้อมูล
2:00-4:00	กลไกเกม	โชว์กระดาน: 5 ทรัพยากร, การ์ด 3 tier, 3 แอคชั่น, ถึง 20 แต้มชนะ
4:00-5:30	ควิซแทรก (จุดเด่นการศึกษา)	สาธิตควิซเด็ง → ตอบถูก+เร็ว = ได้เดินก่อน
5:30-8:00	สถาปัตยกรรม	วาดภาพ 3 ชั้น (ส่วนที่ 2) → อธิบายว่าทำไมแยก Fusion กับ Supabase
8:00-9:30	บอท AI	ตาราง 4 ลำดับ + 4 บุคลิก (ส่วนที่ 4.1)
9:30-11:00	หลังบ้าน & ความปลอดภัย	Matchmaking กัน race / server-authoritative / RLS / OTP
11:00-12:00	ผลทดสอบ	ความพึงพอใจ 4.50, ผู้ทดลอง 15 คน
12:00-13:00	ข้อจำกัด + อนาคต	ตอบเชิงรุก (ส่วนที่ 9) ก่อนอาจารย์ถาม
13:00+	ถาม-ตอบ	ใช้ส่วนที่ 7

เคล็ดลับ: เตรียม เดโมสด หรือ คลิปสำรอง ของ 3 อย่างนี้ให้พร้อม — (1) ชื่อการ์ดได้แต้ม, (2) ควิซเด็งแล้วสลับเทิร์น, (3) จับคู่ออนไลน์ 2 เครื่องเข้าห้องเดียวกัน ถ้าเน็ตมีปัญหาให้เปิดคลิปแทน

? ส่วนที่ 7 — คลังคำถาม-คำตอบ (เตรียมรับมืออาจารย์) ★ ★ ★

นี่คือส่วนสำคัญที่สุด ซ้อมพูดคำตอบให้เป็นธรรมชาติ อย่าท่อง

กลุ่ม A — แนวคิด / เหตุผล (Conceptual)

Q1: ทำไมเลือกเกม Splendor เป็นต้นแบบ?

เพราะกลไกหลักของ Splendor คือ **การบริหารทรัพยากรและการต่อยอด (engine-building)** ซึ่งแมปเข้ากับแนวคิด "การจัดสรรทรัพยากรระบบคอมพิวเตอร์" ได้ตรงมาก — ทรัพยากรมีจำกัด ต้องวางแผนใช้ และการลงทุนช่วงแรกทำให้ช่วงหลังได้เปรียบ (เหมือนโบนัสส่วนลด) อีกทั้งกติกากระชับ เรียนรู้ง่าย เหมาะเป็นสื่อการสอน

Q2: เกมนี้ "สอน" อะไร และวัดผลการเรียนรู้อย่างไร?

สอน 2 ชั้น — ชั้นแรกคือ **แนวคิดทรัพยากรระบบ 5 ด้าน** (CPU/RAM/Network/Storage/Security) ผ่านการเล่นซ้ำๆ จนซึมซับ ชั้นที่สองคือ **ความรู้คอมพิวเตอร์เชิงวิชาการ** ผ่านควิซแทรกและ Daily Quiz ส่วนการวัดผล ใช้แบบสอบถามความพึงพอใจ (Likert 5 ระดับ) กับผู้ทดลอง 15 คน ได้ค่าเฉลี่ย 4.50 อยู่ในระดับมากที่สุด

Q3: ความใหม่/contribution ของงานคืออะไร ในเมื่อ Splendor มีคนทำดิจิทัลแล้ว?

เกม Splendor ดิจิทัลที่มีอยู่ (เช่น Ephod) เน้นความบันเทิงล้วน **ไม่มีเนื้อหาวิชาการ** งานนี้ต่างตรงที่ (1) **ริเริ่มเป็นการศึกษา IT** (2) **ฝังควิซเข้าไปในกลไกเกม** ให้ความรู้กลายเป็นความได้เปรียบ (3) ทำเป็น **ระบบ Full-Stack multiplayer + cloud** ที่ใช้งานได้จริง ไม่ใช่ต้นแบบบนกระดาษ

Q4: ทำไมต้องเป็น "เกม" ทำไมไม่ทำแอปสอน/ควิซตรงๆ?

เพราะใช้หลัก **Gamification** — การแข่งขัน, อันดับ MMR, รางวัลเพชร, การปลดล็อกกรอบ สร้าง **แรงจูงใจภายใน** ให้ผู้เรียนกลับมาทบทวนเองโดยสมัครใจ ซึ่งการอ่านตำราเฉยๆ ทำไม่ได้

กลุ่ม B — เทคนิค (Technical) — อาจารย์สาย CS จะเจาะตรงนี้

Q5: Photon Fusion "Shared Mode" คืออะไร ต่างจาก Host/Server Mode ยังไง ทำไมเลือก?

Shared Mode คือโหมดที่ **ไม่มีเซิร์ฟเวอร์ authority ตัวเดียว** แต่ละ NetworkObject มี "เจ้าของสถานะ" (State Authority) กระจายกัน เหมาะกับเกม session เล็ก ผู้เล่นน้อย และ **ประหยัด** เพราะไม่ต้องรัน dedicated server เอง ผมเลือกเพราะเกมเรา 2-4 คน แต่ผมยังให้ **Master Client ทำหน้าที่เป็นผู้ตัดสิน** เรื่องสุ่มคำถามควิซและสถานะ กระดาน เพื่อกันความขัดแย้งของข้อมูล

Q6: ถ้าผู้เล่น 2 คนกดหาคู่พร้อมกันเป๊ะ ระบบกันการจับคู่ผิดยังไง? (Race Condition)

กัน 2 ชั้น — ชั้นแรกที่ระดับฐานข้อมูลใช้ **SELECT ... FOR UPDATE** ล็อกแถวคิวในธุรกรรม ทำให้ 2 request เข้ามาแก้พร้อมกันไม่ได้ ต้องรอกัน ชั้นที่สองมี **unique partial index** บน player_id เฉพาะแถวที่ status='waiting' ทำให้คนเดียวมีคิวซ้อนไม่ได้เด็ดขาด

Q7: ผู้เล่นโกงคะแนน MMR หรือเพชรได้ไหม?

ไม่ได้ครับ ระบบเป็น **Server-Authoritative** — ตอนจบเกม client ส่งขึ้น server แค่ "อันดับที่จบ + จำนวนผู้เล่น" แล้ว **server คำนวณ MMR/เพชรเองด้วยสูตรเดียวกัน** ที่ใช้โชว์ฝั่ง client นอกจากนี้ตาราง profile ตั้ง **RLS ให้ client เขียนไม่ได้** เขียนได้เฉพาะ Edge Function ฝั่ง server เท่านั้น

Q8: บอทเป็น AI/Machine Learning ไหม?

ไม่ใช่ ML ครับ เป็น **Rule-Based (ระบบกฎเกณฑ์)** ที่มี ลำดับการตัดสินใจ 4 ชั้น บวก heuristic ให้คะแนนการ์ด ผมเลือก Rule-Based เพราะ (1) อธิบายได้ทุกการตัดสินใจ (explainable) (2) **ไม่ต้อง train ไม่ต้องมี dataset** (3) **เบา รันบนมือถือได้ทันที** และ (4) สำหรับเกมกฎตายตัวแบบนี้ Rule-Based ที่ออกแบบดีก็เล่นเก่งพอท้าทายแล้ว ส่วน ML จะ over-engineer เกินจำเป็น

Q9: แล้วบอททำให้มันไม่น่าเบื่อยังไง ในเมื่อเป็นกฎตายตัว?

ผมเพิ่ม **4 บุคลิก (archetype)** ที่ผูกกับตำแหน่งที่นั่ง ปรับน้ำหนัก/เกณฑ์การตัดสินใจต่างกัน เช่น Aggressive เน้นการ์ดแต้มสูง Hoarder ชอบจองเก็บของ และใส่ **ความน่าจะเป็นที่บอทจะเลือกตัวเลือกอันดับรอง** ทำให้ไม่เล่นเหมือนกันเป๊ะทุกตา ดูเป็นมนุษย์ขึ้น

Q10: ทำไมเลือก Supabase ไม่ใช่ Firebase?

เพราะ Supabase ใช้ **PostgreSQL** จริง ผมจึงเขียน **SQL, RPC function, RLS, transaction** ได้เต็มที่ ซึ่งจำเป็นกับการกัน race condition ในระบบจับคู่ อีกทั้งเป็น open-source และ free tier เพียงพอกับโครงการ ส่วน Firebase เป็น NoSQL ทำ logic ระดับ transaction/row-lock ได้ยากกว่า

Q11: ข้อมูลกระดานของแต่ละเครื่องเห็นไม่ตรงกันจะอย่างไร?

ผมให้ **Master Client** เป็น **authority** แล้ว broadcast snapshot ของกระดาน/เหรียญ/เทิร์น และเจอปัญหาจริงว่าคนเข้าห้องช้าจะ "พลาด" broadcast เพราะฉาวยังโหลดไม่เสร็จ ผมเลยแก้เป็นให้ client ที่พร้อมแล้ว **"ขอสถานะเต็มจาก host เอง" (pull)** แทนการรอรับ (push) ทำให้ late-joiner sync ตรงเสมอ

Q12: ออกแบบฐานข้อมูลก็ตาราง มีความสัมพันธ์ยังไง?

8 ตารางหลัก ตัวอย่างความสัมพันธ์: `player_profiles` ผูก 1:1 กับ `auth.users`, `matchmaking_queue` อ้างถึง `rooms` (FK), `quiz_questions` อ้างถึง `quiz_patches` (FK แบบ cascade) ออกแบบให้ข้อมูลถาวร (โปรไฟล์/คะแนน) แยกจากข้อมูลชั่วคราว (คิว/ห้อง) ชัดเจน

Q13: ระบบควิซแก้ไขข้อสอบต้อง build เกมใหม่ไหม?

ไม่ต้องครับ ผมทำเป็น **Patch System** — คำถามเก็บใน `quiz_questions` ผูกกับ `quiz_patches` ตัวเกมดึงเฉพาะ patch ที่ `is_active=true` ผ่าน RPC ดังนั้นแก้ไขข้อสอบในฐานข้อมูลแล้วเกมอัปเดตเองทันที (นี่เป็นฐานสำหรับต่อยอดเป็น Instructor Dashboard ในอนาคต)

กลุ่ม C — การออกแบบ/ความสมดุล (Design)

Q14: ทำไมตั้งเป้าหมายที่ 20 แต้ม? ทำไมควิซทุก N รอบ?

เป็นค่าที่ได้จาก **การปรับสมดุล (balancing)** จากการทดลองเล่น ให้เกมจบในเวลาที่กำลังดี ไม่สั้นไม่ยืดเกิน ส่วนความถี่ควิซก็ดูให้แทรกความรู้ได้สม่ำเสมอโดยไม่ขัดจังหวะเกมจนน่ารำคาญ (ดู  ส่วนที่ 8 เรื่องตัวเลขควิซในโค้ดก่อนตอบ)

Q15: ทำไม MMR ใช้สูตรแบบ DOTA2 ไม่ใช่ Elo?

Elo ออกแบบมาสำหรับ **1v1** และคิดจากส่วนต่างเรตตั้งคู่แข่ง แต่เกมเราเป็น **2-4 คน จัดอันดับ** ผมเลยใช้สูตรแบบให้คะแนน/หักตามอันดับที่จบ (placement-based) แบบ DOTA2 ซึ่งเข้าใจง่ายและเหมาะกับ multiplayer มากกว่า

กลุ่ม D — ข้อจำกัด (Limitations)

Q16: จุดอ่อนของระบบคืออะไร?

3 เรื่องหลัก (1) **บอทเป็นกฏตายตัว** ยังไม่ปรับความยากตามฝีมือผู้เล่นอัตโนมัติ (2) **ยังไม่มีหน้าจัดการข้อสอบสำหรับอาจารย์** ต้องแก้ผ่านฐานข้อมูลโดยตรง (3) **ยังไม่มีระบบ chat/emoji** ระหว่างผู้เล่น — ทั้ง 3 ข้อนี้ผมระบุไว้เป็น **งานในอนาคต** แล้ว

Q17: ทดสอบกับผู้ใช้จริงกี่คน เพียงพอไหม?

ทดลองกับนิสิต 15 คน ผ่านแบบสอบถาม Likert 5 ระดับ ได้ค่าเฉลี่ย 4.50 (S.D. 0.54) ซึ่งเพียงพอสำหรับการประเมินเชิงความพึงพอใจในระดับโครงการ แต่ผมยอมรับว่าถ้าจะวัด **ผลสัมฤทธิ์การเรียนรู้** จริงๆ ควรทำ pre-test/post-test กับกลุ่มตัวอย่างใหญ่ขึ้น ซึ่งเป็นข้อเสนอแนะสำหรับงานต่อไป

⚠️ ส่วนที่ 8 — จุดที่เอกสารกับโค้ดไม่ตรงกัน (แก้ก่อนสอบ!) ★ สำคัญมาก

ผมตรวจโค้ดจริงเทียบกับเล่มเอกสาร เจอ **5 จุดที่ไม่ตรงกัน** ถ้าอาจารย์เปิดโค้ดดูหรือเล่นเกมจริงจะเห็นไม่ตรงกับที่เขียน แนะนำให้เลือกทางใดทางหนึ่ง: **แก้โค้ดให้ตรงเล่ม** หรือ **แก้เล่มให้ตรงโค้ด** ก่อนวันสอบ

#	เรื่อง	เล่มเขียนว่า	โค้ดจริงเป็น	คำแนะนำ
1	ความถี่ควิซ	ทุก 3 รอบ (Abstract, 3.6, 7)	<code>quizInterval = 5</code> → ทุก 5 รอบ	แก้โค้ดเป็น 3 ง่ายสุด (แก้ 2 บรรทัด)
2	เวลาตอบควิซ	20 วิ (Scope 3.5)	<code>timeLimit = 10f</code> → 10 วิ	แก้เล่ม 3.5 เป็น 10 วิ (บทพล 6.4 ก็เขียน 10 วิอยู่แล้ว)
3	บอทดอบควิซถูก	100% (Scope 3.5)	สุ่ม 40–80%	แก้เล่มเป็น 40–80% (ดีกว่า เพราะ 100% ผู้เล่นชนะเทิร์นไม่ได้)
4	จังหวะบอทดอบ	"5 วิสุดท้าย" (3.5, 7.2)	สุ่ม 1–10 วิ ตลอดช่วง	แก้เล่มเป็น "สุ่มทั้งช่วง" หรือแก้โค้ดให้ตอบช่วงท้าย
5	เงื่อนไขจบเกม	เล่นจบรอบก่อน (ให้เดินเท่ากัน) — Scope 3.4	จบทันทีที่ถึง 20 แต้ม	ตัดสินใจ: ถ้าจะคงโค้ด ให้แก้เล่ม 3.4

รายละเอียดแต่ละจุด (เพื่ออาจารย์ถามเจาะ):

- **#1 ควิซทุก 5 รอบ:** ไฟล์ `GameController.cs` ตั้ง `quizInterval = 5` และ `onlineQuizTurnInterval = 5` เงื่อนไขใน `GameController.Turns.cs` คือ `(รอบ-1) % 5 == 0` → ควิซจะแจ้งเตือนเริ่มรอบที่ 6, 11, 16 (บวกควิซรอบแรกตอนเปิดเกม) **ถ้าต้องการ "ทุก 3 รอบ" ตามเล่ม เปลี่ยน 5 เป็น 3 ทั้งสองตัว**
- **#3 + #4 พฤติกรรมบอทควิซ:** อยู่ในฟังก์ชัน `SimulateOtherPlayers` (`QuizManager.cs`) — `botDifficulty = Random.Range(0.4f, 0.8f)` (โอกาสตอบถูก 40–80%) และ `botSpeed = Random.Range(1.0f, timeLimit)` (ตอบที่วินาทีสุ่ม 1–10) **หมายเหตุ:** บทคัดย่อ (Abstract) เขียนว่า "randomized quiz response delay" ซึ่ง **ตรงกับโค้ด** อยู่แล้ว มีแค่ Scope 3.5 กับ Discussion 7.2 ที่ขัดกัน
- **#5 จบเกมทันที:** `CheckWinCondition` ถูกเรียก **ทุกจบเทิร์น** ถ้าใครถึง 20 จะ `isGameOver = true` แล้วจบเลย (มีคอมเมนต์ในโค้ดระบุว่าตั้งใจเปลี่ยนจาก "เช็คตอนจบรอบ" มาเป็น "เช็คทันที") **ถ้าอาจารย์ถามเรื่องความยุติธรรม (คนเดินก่อนได้เปรียบ)** ให้ตอบว่าเป็น first-to-20 ที่จบทันทีเพื่อความชัดเจน หรือจะแก้กลับให้จบรอบตามเล่มก็ได้

 **ถ้ามีเวลาแก้แค่อย่างเดียว แก้ #1** (เปลี่ยน 5→3) เพราะเป็นตัวเลขที่อาจารย์เห็นง่ายสุดตอนเล่น และแก้ในโค้ดแค่ 2 บรรทัด ส่วน #2–#5 ถ้าไม่ทันแก้ ให้ **จำไว้และตอบตามโค้ดจริง** อย่าตอบตามเล่มที่ผิด

🌟 ส่วนที่ 9 — ข้อจำกัด & งานในอนาคต (ตอบเชิงรุกให้ดูโตกว่าคำถาม)

พูดเรื่องนี้เองในสไลด์ก่อนอาจารย์ถาม จะดูเป็นมืออาชีพ (ดึงจาก Discussion 7.4):

1. **Dynamic Difficulty** — ทำให้บอทปรับความเก่งตามฝีมือผู้เล่นอัตโนมัติ
 2. **Instructor Dashboard** — หน้าเว็บให้อาจารย์จัดการคลังข้อสอบเองผ่านคลาวด์ (ต่อยอดจาก patch system ที่มีอยู่แล้ว)
 3. **ระบบโต้ตอบทางสังคม** — Preset Emoji / ข้อความสำเร็จรูป ในห้องรอและระหว่างเล่น
 4. **(เสริมได้)** วัดผลการเรียนรู้ด้วย pre-test/post-test กับกลุ่มตัวอย่างใหญ่ขึ้น
-

📖 ส่วนที่ 10 — อภิธานศัพท์ (ต้องนิยามได้ถ้าโดนถาม)

คำ	นิยามสั้นๆ ที่พูดได้
Engine-Building	กลไกเกมที่การลงทุนช่วงต้น (ซื้อการ์ด→ได้โบนัส) ทำให้ช่วงหลังทำแต้มเร็วขึ้นเรื่อยๆ
Photon Fusion	SDK เครือข่ายเรียลไทม์สำหรับเกม Unity ทำงานบน UDP
Shared Mode	โหมด Fusion ที่ไม่มี server กลางตัวเดียว, authority กระจายตาม object
Master Client	ผู้เล่นที่ถูกเลือกให้เป็น "ผู้ตัดสิน" สถานะรวมในห้อง
State Synchronization	การทำให้สถานะเกมบนทุกเครื่องตรงกัน
Race Condition	สภาวะที่หลายคำสั่งทำงานพร้อมกันแล้วผลลัพธ์ผิดเพี้ยน
SELECT FOR UPDATE	คำสั่ง SQL ล็อกแถวข้อมูลในธุรกรรม กันแก๊วพร้อมกัน
Server-Authoritative	server เป็นผู้คำนวณ/ตัดสินค่าจริง client ปลอมไม่ได้
RLS (Row Level Security)	กฎความปลอดภัยระดับแถวใน PostgreSQL คุมว่าใครอ่าน/เขียนแถวไหนได้
BaaS	Backend-as-a-Service — บริการหลังบ้านสำเร็จรูปบนคลาวด์ (เช่น Supabase)
RPC / Database Function	ฟังก์ชันที่รันฝั่ง server/ฐานข้อมูล เรียกจาก client ได้
MMR	Matchmaking Rating — คะแนนวัดฝีมือใช้จัดอันดับ/จับคู่
OTP	One-Time Password — รหัสยืนยันใช้ครั้งเดียว (เราเก็บเป็น hash)
Heuristic	เกณฑ์/สูตรประมาณการช่วยตัดสินใจ (ในบอทใช้ให้คะแนนการ์ด)

✅ เช็กลิสต์ก่อนเข้าห้องสอบ

- ☐ ท่องตัวเลขในส่วนที่ 1 ได้คล่อง (20 แต้ม, 5 ทรัพยากร, 3 tier, 10 เหรียญ, 2–4 คน)
- ☐ ข้อสอบ Q5–Q13 (คำถามเทคนิค) ให้เป็นธรรมชาติ
- ☐ **ตัดสินใจเรื่องส่วนที่ 8** (จะแก้โค้ด แก้เล่ม หรือจำไว้ตอบ) อย่างน้อยข้อ #1
- ☐ เตรียมเดโม/คลิปสำรอง 3 อย่าง (ซื้อการ์ด, คริวสลับเทิร์น, จับคู่ออนไลน์)
- ☐ เปิดสไลด์ข้อจำกัด+อนาคต (ส่วนที่ 9) ไว้ตอบเชิงรุก

ขอให้พีรเซ็นต์ผ่านฉลุยครับ! 🙏 ถ้ามีคำถามเพิ่มหรืออยากให้ซ่อมตอบข้อไหน กลับมาถามได้เลย